SDI 4213-980: DevOps – CI/CD

Week 7 Individual Exercise: Multi-Service Help Desk Application

Exercise Goal

In this individual exercise, you will take a working browser-based Service Request Tracker and convert it into a multi-service Docker Compose application. Your final system will include a Streamlit web application, PostgreSQL database, Adminer database interface, Docker networking, environment configuration, a persistent volume, and service health checks.

This is an individual assignment. Your screenshots and repository history must show your own work.

What You Will Build

```
Browser → Streamlit web service → PostgreSQL database → named volume
                    ↘ Adminer database interface
```

Required Software

- Git and GitHub
- VS Code
- Python 3.13 or compatible Python 3
- Docker Desktop with Docker Compose
- A web browser

Starter Repository

Use the instructor-provided Week 7 starter repository. The application already runs locally using SQLite. Your task is to configure and prove the Docker Compose version using PostgreSQL.

Milestone 1 – Confirm the Starter App Works

1. Clone your Week 7 repository.
2. Create and activate a virtual environment.
3. Install requirements.
4. Run the automated tests.
5. Start the Streamlit app locally.

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m pytest -v
streamlit run app/main.py
```

Create at least one service request in the browser.

SCREENSHOT 1 REQUIRED: Show the browser application running locally with at least one request visible. Include enough of the browser window to show the application title and request table.

Milestone 2 – Prepare Environment Configuration

6. Copy `.env.example` to `.env`.
7. Change the development password to a value you choose.
8. Confirm `.env` is excluded by `.gitignore`.
9. Do not commit `.env`.

```
POSTGRES_USER=student
POSTGRES_PASSWORD=<your-development-password>
POSTGRES_DB=helpdesk
```

Milestone 3 – Build the Compose Application

Complete the provided `compose.yaml`. It must contain three services:

- `web` – build the Streamlit application from the Dockerfile and publish port 8501.
- `db` – use PostgreSQL and configure it from environment variables.
- `adminer` – provide a browser interface to inspect PostgreSQL and publish port 8080.

Also create a named volume for PostgreSQL data and configure the web service to connect to the host `db`, not `localhost`.

Milestone 4 – Add Database Health and Dependency Ordering

Configure a PostgreSQL health check using `pg_isready`. Configure the `web` and `adminer` services so they wait for the database service to be healthy.

Milestone 5 – Start the Multi-Service Stack

```
docker compose up --build -d
docker compose ps
```

All three services should be running. The database should report healthy.

SCREENSHOT 2 REQUIRED: Show `docker compose ps` with the web, db, and adminer services running and the database healthy.

Milestone 6 – Use the Help Desk Application

Open `http://localhost:8501`. Create at least two distinct service requests and update the status of one request.

SCREENSHOT 3 REQUIRED: Show the Help Desk dashboard with at least two requests and visible status information.

Milestone 7 – Inspect PostgreSQL with Adminer

Open `http://localhost:8080`. Connect to PostgreSQL using the values from your `.env` file. Use `db` as the server/host name. Locate the `requests` table and view its rows.

SCREENSHOT 4 REQUIRED: Show Adminer displaying the `requests` table or its records. Do not expose your database password in the screenshot.

Milestone 8 – Demonstrate Persistence

Stop and remove the application containers without deleting the volume:

```
docker compose down
docker compose up -d
```

Reload the Help Desk application and verify that the requests you created are still present.

SCREENSHOT 5 REQUIRED: Show the restarted application with the same previously created requests still visible. Your screenshot should demonstrate that the data survived container replacement.

Milestone 9 – Inspect Logs

```
docker compose logs web
docker compose logs db
```

SCREENSHOT 6 REQUIRED: Show useful web-service log output. Your screenshot should include the command and several log lines.

Milestone 10 – Demonstrate the Effect of Removing the Volume

Before continuing, make sure Screenshots 3–6 are complete. Then intentionally remove the application and its named volume:

```
docker compose down -v
docker compose up -d
```

Reload the Help Desk application. The previously stored requests should no longer be present because the database volume was deleted.

SCREENSHOT 7 REQUIRED: Show the restarted application after volume deletion and explain in the caption what changed and why.

Milestone 11 – Git/GitHub Workflow

Complete the work using the standard course workflow:

```
Issue → Branch → Change → Commit → Push → Pull Request → CI → Review → Merge
```

Your branch should include `compose.yaml`, any safe documentation changes, and your evidence file. Do not commit `.env`.

SCREENSHOT 8 REQUIRED: Show your final GitHub pull request with a passing CI check. The repository/branch context should be visible.

Screenshot Submission Requirements

Screenshot	Evidence
1	Local Streamlit app running
2	All Compose services running / database healthy
3	Help Desk dashboard with two or more requests
4	PostgreSQL records visible through Adminer
5	Requests persist after down/up
6	Web-service logs
7	Effect of docker compose down -v
8	Final GitHub pull request with passing CI

For every screenshot, add a short caption describing what the screenshot proves. Screenshots must be readable and must not contain passwords, tokens, API keys, or other secrets.

Required Deliverables

- GitHub repository URL
- Completed compose.yaml
- Updated .env.example if needed
- Evidence document with all 8 screenshots and captions
- Final pull request showing passing CI
- Short reflection answering the questions below

Reflection Questions

10. Why must the web service use `db` rather than `localhost` to reach PostgreSQL?
11. What problem does the named Docker volume solve?
12. What is the difference between `docker compose down` and `docker compose down -v`?
13. What role does the database health check play?
14. How is Docker Compose similar to Infrastructure as Code?
15. What was the most difficult problem you encountered and how did you solve it?

Suggested Grading Rubric

Category	Points
Local starter validation and tests	10
Compose web + PostgreSQL + Adminer configuration	25
Networking and environment configuration	15
Named volume and persistence demonstration	15
Health check / dependency ordering	10
Screenshot evidence	15
Git/GitHub workflow and passing CI	5
Reflection and documentation	5
Total	100